

ADR 0099 — A preview is real git, refusing rather than modelling

Status: Accepted — server half implemented, `POST /api/preview` routed; the web canvas (A6) is a follow-up **Date:** 2026-08-30
Issues: [#576](#) — M10.08, graph preview **Spec:** `docs/superpowers/specs/2026-08-29-graph-preview-design.md`
(amended 2026-08-30: §4.1b/4.1c placement, §5 comparison, §9b open questions) **Supersedes:** nothing · **Superseded by:** nothing

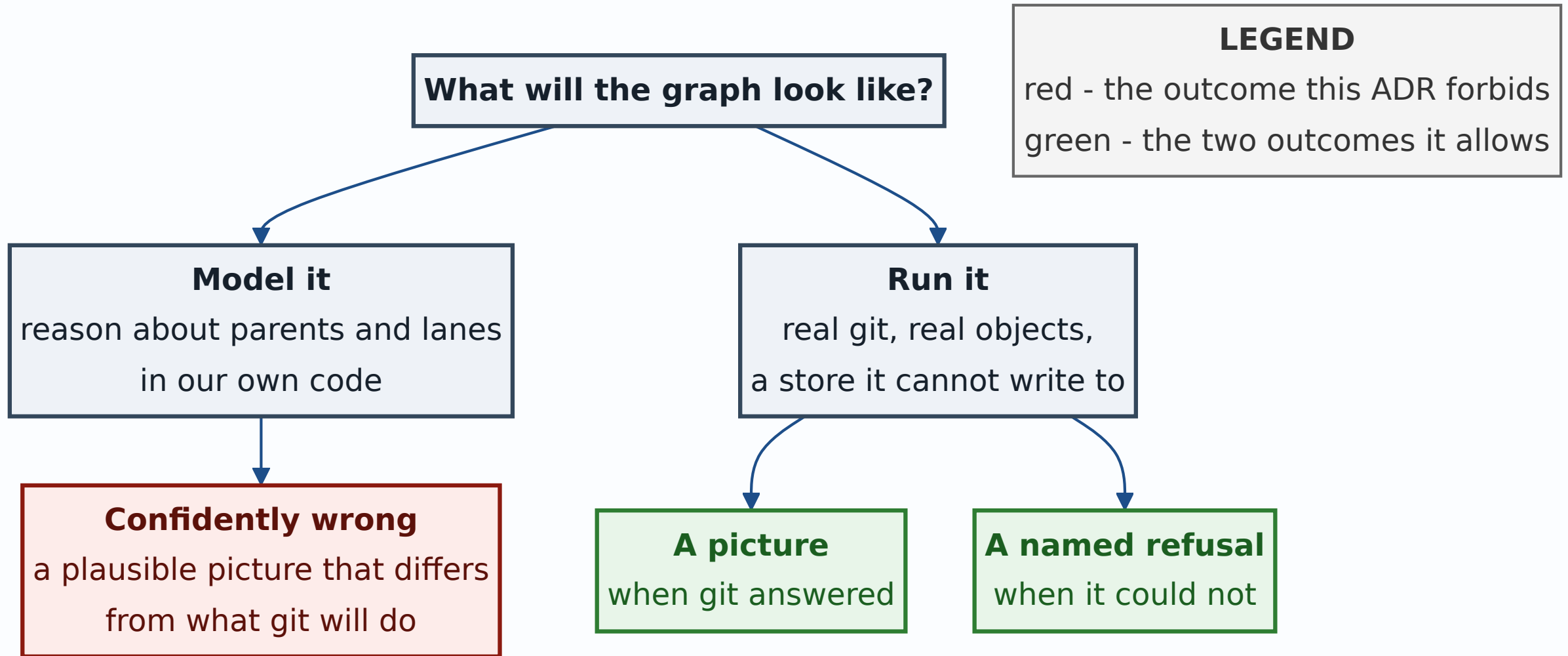
Context

A user about to run `revert`, `cherry-pick` or a merge cannot picture what the graph will look like afterwards. Explain Mode already says what an operation will do, in words. Nothing shows it.

There are two ways to build that picture, and only one of them is safe.

The tempting way is to **model** git: reason about parents and lanes in our own code and draw the result. The failure mode of a modelled git is not being wrong. It is being *confidently* wrong — producing a plausible graph that quietly differs from what the command will actually do, on exactly the operations a user cannot check by eye. A user who sees nothing goes and reads the docs. A user who sees a wrong picture acts on it.

The other way is to **run** git and refuse when it cannot answer. That is what this ADR decides.



The mechanism that makes "run it" safe was measured before it was designed, and re-measured on `8ef604d1` on 2026-08-30 with the exact argv composition the code uses. A throwaway **bare** repository whose `objects/info/alternates` names the served repository's object directory can read every object that repository has and writes only into itself:

```
objects under <commdir>/objects before : 9
merge-tree -z --write-tree --merge-base=... : rc=1 (a real conflict)
objects under <commdir>/objects after    : 9      <- unchanged
objects under <scratch>/objects          : 3
commit-tree <tree> -p <head>              : rc=0, commit 6b205017...
objects under <commdir>/objects after    : 9      <- still unchanged
scratch store:    cat-file -t 6b205017... -> commit
real repository:  cat-file -t 6b205017... -> fatal: could not get c
real repository:  show-ref                 -> byte-identical before
```

Decision

A preview is computed by real git against the real object store, in a throwaway store it can read through and cannot write back to, and it refuses rather than models.

Six parts, each load-bearing.

1. The computation lives in `git-vista-server`, not `git-vista-git`

`git-vista-git` is a pure-`gix` crate that never spawns a process; it carries its own `ALLOWED_GIT_CRATE_SPAWN_SITES` allowlist precisely to keep it that way. The sanctioned `git merge-tree --write-tree` path already exists one crate over, in `activity::revert_would_conflict` (#327), already allowlisted, already going through the sealed sandbox launcher. `crates/git-`

`vista-server/src/preview.rs` constructs **no** `Command`: every spawn goes through `git_cmd::git_output`, so `argv_boundary`'s source scan needs no new entry and its allowlist is untouched.

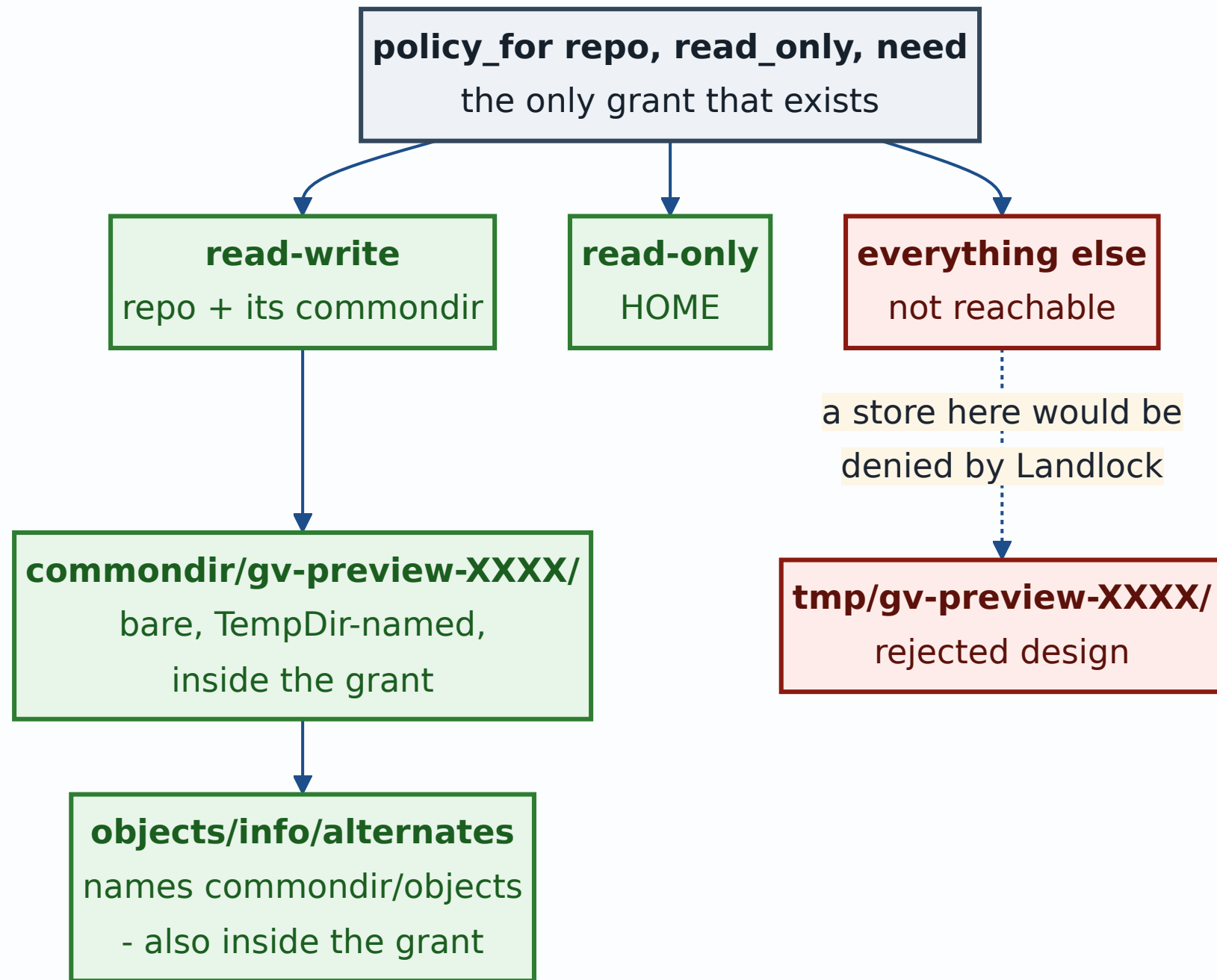
2. The scratch store lives under `<commdir>`, and nowhere else

`git_cmd::sandboxed` derives `read_only` from `state::read_only_for_path(repo)` and calls `sandbox::policy_for(repo, read_only, need)`, which grants exactly `repo` and its resolved `commdir` read-write plus `$HOME` read-only. Nothing else on the filesystem is reachable by the child.

A store in `/tmp` would be created fine and then fail on its own `objects/info/alternates`, which names a path outside every grant — Landlock denies the read and the preview fails for a reason that has nothing to do with git. Inside `commdir` there is exactly one grant and no new policy.

The spawn passes the **real repository** as `repo`, so the grant is built from it, and selects the store with `--git-dir=<abs>`.

`sandbox::network_need` skips that as a bare flag and classifies `merge-tree/commit-tree/show` as `NetworkNeed::Local`. **No security-boundary change; nothing under `sandbox/` was edited.**



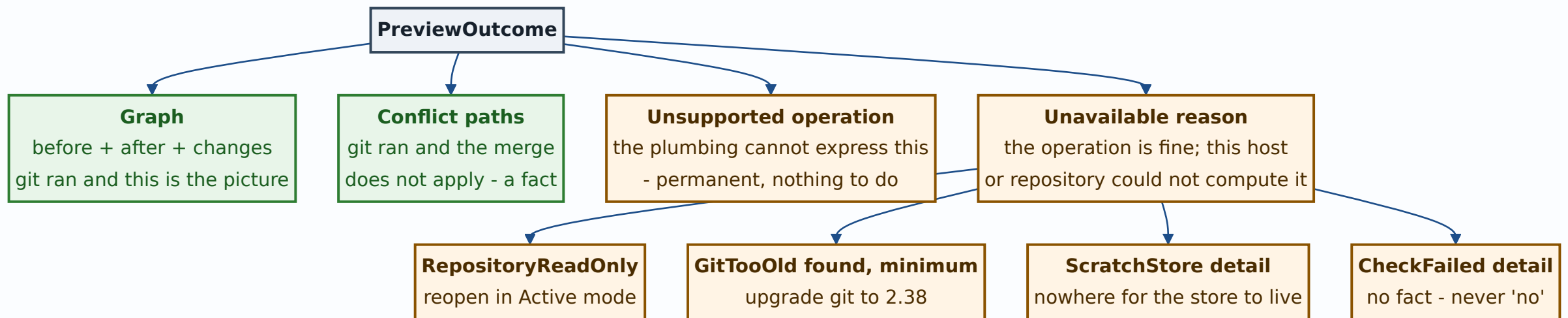
3. A2's guarantee is narrower than "the repository is untouched" — say so

A2 is "no new object under `<commdir>/objects`", not "nothing written under `.git`". The scratch store *is* a real directory created inside `commdir`. A test that counted files under `<commdir>` would count the store's own objects and go red for exactly the reason the design works. The acceptance test therefore counts under `<commdir>/objects` specifically, compares every ref before and after, and asserts no `gv-preview-*` directory survives.

Cleanup is `tempfile::TempDir`'s `Drop`, which covers the return, the `?` and the panic — but **not** `SIGKILL` or a power loss. So `ScratchStore::new` sweeps stale `gv-preview-*` siblings older than an hour first. The sweep only ever considers directories whose name carries the prefix this module chose, and only removes ones older than the bound; an entry whose age cannot be read is left alone. "We could not tell how old it is" is not grounds to delete something inside someone's repository.

The prefix is named rather than `tempfile`'s default `.tmpXXXXXX` for a reason worth stating: a sweep matching a prefix nothing produces is **inert**, and a test that hand-created a stale directory would pass anyway.

4. Four arms, because there are four different things to say



`Unavailable` is a **fourth arm**, not a `reason` field bolted onto `Unsupported`. Folding them together would make one variant mean two different things — "this operation can never be previewed" and "this operation is fine, this repository cannot host the computation" — which is exactly the shape `plan.rs` rejects by name in `RevertCommit`'s doc comment ("Modelling it as `RevertCommit { commit, mainline: Option<u8> }` would have made `None` mean two different things ... Two variants make the second unrepresentable instead of merely checked"). The two also demand different things of the reader: `Unsupported` → nothing to do, ever; `Unavailable { RepositoryReadOnly }` → reopen in Active mode and it works.

The shape is **borrowed, not invented**. `recovery_center.rs` already draws this exact distinction: `Expired { WouldConflict }` is "a live check ran and returned a definite negative — a fact, not a guess", while `CheckFailed { detail }` is "the live check itself could not run. 'No fact', never 'no'." `Conflict { paths }` here is the `Expired` shape; `Unavailable` is the `CheckFailed` shape.

`Conflict { paths: [] }` is **not representable as an answer**: git reporting a conflict while we could name no file reads as "conflicted, nothing conflicted", so it is `CheckFailed`. That rule earned its keep during mutation testing — deleting the alternates write makes `merge-tree` exit 1 with nothing parseable, and this is what turns that into a named failure instead of a silent empty conflict.

5. The checks run in a pinned order

1. `Unsupported` — pure, from `plan.operation` alone, no IO. First because it is the permanent answer: telling someone to reopen in Active mode does not help an operation that can never be previewed.
2. `RepositoryReadOnly` — one catalog lookup, still no spawn.

- 3. `GitTooOld` — the cached version probe.
- 4. The git work.

So a read-only repository on an old git reports the read-only fact, which is the one the user can act on.

6. `Unsupported` is the default arm

One `match` on `GitOperation` exists in the whole module, in `previewable`, and its last arm is `_ => None`. A variant added to the protocol later is invisible here rather than wrong. The name reported to the user is read from `serde`'s own `"op"` tag, so a later variant is named correctly without anyone editing this file.

Three operations are supported and two named ones are deliberately not:

Operation	Base	Ours	Theirs	Parents
<code>RevertCommit { commit }</code>	<code>commit</code>	HEAD	<code>commit's</code> sole parent	<code>[HEAD]</code>
<code>CherryPick { commit }</code>	<code>commit's</code> sole parent	HEAD	<code>commit</code>	<code>[HEAD]</code>
<code>MergeBranch { branch }</code>	<i>git</i> <i>computes it</i>	HEAD	tip	<code>[HEAD,</code> <code>tip]</code>
<code>CherryPick Merge / RevertMerge</code>	—	—	—	Unsupported
<code>rebase / reset / force-push</code>	—	—	—	Unsupported

A merge commit or a root commit has no *sole* parent, and `merge-tree` needs one as `theirs` — so reverting or picking one is `Unsupported` too, at the instance level. Same fail-closed rule `activity::undoables` already applies.

The revert row is byte-identical to the merge `activity::revert_would_conflict` already runs, which is what makes this preview and the app's own revert offer consistent by construction rather than by review.

The three questions §9b left open, answered

1. A read-only repository

`PreviewOutcome::Unavailable { reason: PreviewUnavailable::RepositoryReadOnly }`. Detected with `state::read_only_for_path(repo)` — the **same** function `git_cmd::sandboxed` derives `read_only` from before calling `sandbox::policy_for`. One source of truth means the refusal and the Landlock grant can never disagree.

Honest limit: `read_only_for_path` answers `false` for a path with no catalog entry, so an unregistered but genuinely unwritable directory reports `ScratchStore { detail }` rather than `RepositoryReadOnly`. That is the same answer the sandbox gives it, so nothing is laundered — and both paths are tested, so the read-only arm cannot rot unnoticed.

`POST /api/preview` deliberately does **not** answer 403 for a read-only repository, unlike every other write-posture route. Refusing there would make this arm unreachable in production and exercisable only from a test, which is how a named reason rots into decoration.

2. `color` and `on_remote` for a commit that does not exist

`on_remote` is `false` **on every row of both halves**, because `git_vista_core::preview::lay_out_preview` is `layout_with_refs` and nothing else — the server's remote-membership stamping pass is deliberately not part of that pipeline. Stamping would make the preview-versus-reality comparison red on its own: the throwaway copy the real half is laid out from is a different repository object-for-object only in its new commit.

`color` comes from `layout_with_refs`'s own assignment, which is why the `ref_moves` list is a **precondition and not a decoration**. `layout_with_refs` reserves lane 0 and seeds colour slot 0 from the ref slice it is handed, so a `ref_moves` entry that matched nothing puts the hypothetical commit in lane 1 with a synthetic colour — a confidently wrong picture drawn from correct data.

`lay_out_preview` reports both ways a caller can get that wrong, and this module treats **either** as `CheckFailed` rather than returning the damaged graph. Returning it is not an option.

Consequence a UI must know: **preview lane numbers do not match the live graph's**. The preview lays out one capped `walk_history(repo, 500)`; the live view is a windowed, cursor-paginated walk. Both are internally consistent, and the parity test only ever compares preview-pipeline to preview-pipeline — but a surface that puts the two side by side will show the same commit in different columns. That is why `PreviewOutcome::Graph` carries **both** halves and never `after` alone.

3. The git-version floor

`merge-tree --write-tree` needs git $\geq$ 2.38. The product floor is **2.32 and stays 2.32** — `docs/SUPPORTED_VERSIONS.md` says so, CI builds and exercises a real 2.32 binary (#365, ADR 0082), and a

host on 2.32–2.37 is a fully supported host on which everything else works. So this is one feature's floor and it **degrades**, in the body, to `GitTooOld { found, minimum }`.

The probe runs **per process, lazily, on the first preview** — a `tokio::sync::OnceCell` in `preview.rs`.

- **Not beside the boot probe.**

`sandbox::probe::run_at_startup()` 's own comment states its contract: "There is no degrade: a verdict other than `Contained` means no server, full stop (ADR 0029)." That gate has exactly one non-fatal outcome and must stay that way. Putting a non-fatal *capability* question into a fatal gate is how a degrade gets bolted onto a gate whose whole argument is that it has none.

- **Not per call.** The git binary a process execs is a property of that process's `PATH`, not of the repository or the request.
- **Not inferred from `merge-tree`'s exit code.** Below 2.38 the flag is rejected with exit 128 and a usage string — indistinguishable, by exit code, from a genuinely bad revision, which is the `_ => Err(stderr)` arm `revert_would_conflict` already has. Distinguishing them from `stderr text` is exactly the heuristic that function's doc comment rejects ("that contract — unlike git's prose — doesn't shift with locale or version"). An explicit probe converts an unnameable failure into a named one.

Only a success is cached, so a transient failure to run git does not permanently disable the feature. Honest limit, stated rather than hidden: an operator who upgrades git under a running server does not get this feature until restart. That is the posture `sandbox::capabilities::current()` already takes.

Alternatives considered

Model the graph in the frontend

Rejected before anything else. #457 commits the terminal UI to drawing "from the lanes core already computes"; a preview computed in wasm could never serve that surface, so it would be written twice and drift twice. And it is a model, which §4.3 of the spec rules out on its own.

Put `preview()` in `git-vista-git`

The original design said this, **and it was wrong against the code** — corrected in the spec on 2026-08-30. `git-vista-git` is pure `gix` and never spawns; it carries an allowlist to keep it that way. Moving the spawn there would have been a real boundary change dressed up as a file placement.

A scratch store in `/tmp`

Refused by Landlock at its own `alternates`, as §2 records. `policy_for` does not enforce the managed root — its doc comment says so, and names the reason — so pointing a spawn at a scratch directory is not refused *per se*; the store would be created and then fail to read. Measured, not assumed.

A `reason` field on `Unsupported` instead of a fourth arm

Rejected: one variant meaning two things, the shape `plan.rs` refuses by name. See §4.

Pass `--merge-base` for `MergeBranch` too

This is a deliberate deviation from the written contract, which specified `merge_base: String` with `base = merge-base(HEAD, tip)`. `git merge` computes its own base with the recursive strategy, which builds a **virtual** merge base when two branches

have more than one. Passing a single `git merge-base` answer would produce a tree `git merge` would not produce on a criss-cross history — a confidently wrong picture, the one failure §4.3 exists to make impossible. So `Recipe::merge_base` is an `Option`, `None` for `MergeBranch`, and `git` does what `git` would do. Revert and cherry-pick *are* synthetic three-way merges with a stated base, so they still pass one.

Two `merge-base --is-ancestor` spawns to detect fast-forward

Unnecessary. `merge-base(head, tip)` equals `tip` exactly when `tip` is an ancestor of `head` (already up to date) and equals `head` exactly when `head` is an ancestor of `tip` (fast-forward). One spawn answers all three questions. Both cases must be caught **before** any `commit-tree` call, or the preview invents a merge commit `git` would never write.

Pin `GIT_AUTHOR_DATE` / `GIT_COMMITTER_DATE` on the hypothetical commit

Not possible: `git_cmd` exposes no arity that adds an environment variable, and widening the sealed launcher for a preview is not a trade worth making. So the hypothetical commit's oid is **not reproducible, by construction**, and nothing downstream may compare it by identity. The parity test maps it onto the real commit by position instead.

Consequences

- **A2's guarantee is exactly "no new object under `<commdir>/objects`",** and a scratch *directory* does appear under `commdir` for the life of a call. Anyone tightening that assertion must count under `objects`, not `commdir`.

- `git init --bare` copies `init.templateDir`, so `hooks/` would arrive populated with git's 14 `.sample` files. Inert today — `merge-tree` and `commit-tree` fire no hooks — but `policy_for` sets `HookMode::Run` regardless, so `-c init.templateDir=` is pinned **now**, not the day a future step here fires a hook. Measured: with the flag, the store has no `hooks` directory at all.
- **The object format is inherited from the served repository.** Measured: a `--object-format=sha1` store answers `fatal: Not a valid object name` for a SHA-256 repository's own HEAD, while `--object-format=sha256` answers `commit`. Only the hash format crosses an alternates boundary, which is why it is the only thing inherited; `refStorage` needs no matching because this store never holds a ref.
- **A whole-product limitation, found while testing that:** `git-vista-git` opens every repository with `gix`, and `gix` refuses a SHA-256 repository outright — `read_refs` answers `Open { message: "Failed to load the git configuration" }`. So `preview()` end-to-end answers `Unavailable { CheckFailed }` on such a repository today, and the object-format inheritance is correct-but-not-yet-reachable. That is `git-vista-git`'s to fix, not this module's, and it is recorded here rather than papered over.
- **Seven git spawns per preview** for a revert or cherry-pick, eight for a merge, plus one more on the first call of the process — each through `bwrap` and the shim. (`--version` once, `rev-parse --show-object-format`, `init`, `rev-parse HEAD`, `show -s` on the named commit or `merge-base`, `merge-tree`, `commit-tree`, `show -s` read-back.) Fine for a user-initiated preview; **not** fine per keystroke or per row. A surface that wants it live needs its own caching decision.
- **POST /api/preview sits at the full write posture** (`SessionAndCsrf`, loopback router only) despite mutating nothing: `security.rs` keys on the HTTP method, and a LAN visualize session must never see the plan-review surface (ADR

0005). `planner/contract_suite.rs` classifies it as a non-git-write and pins the inverse requirement — the handler must reach `preview()` and must reach no execution entry point at all.

- **The merge message omits git's `into <branch>` clause.**

Cosmetic: the parity test compares parent topology, lane and row order, never message text.

Decision log

- Placement moved from `git-vista-git` to `git-vista-server` after reading the crate, not after a test failure — the original spec claim was false.
 - Scratch store placed under `comondir` after measuring `policy_for`'s grants, not after a Landlock denial.
 - `Conflict { paths: [] }` made unrepresentable *before* it was needed; the alternates mutation later proved it was the thing that turned a silent empty conflict into a named failure.
 - `merge_base` widened to `Option` against the written contract, for the criss-cross fidelity reason above. Named as a deviation.
 - `%ct` chosen over the contract's `%at` after reading `git_vista_git::history`'s `time: info.commit_time()`. Author time would have put the hypothetical row on a different clock from every other row in the same graph, and `stable_topo_order` sorts on that field.
 - The scratch prefix was made explicit (`gv-preview-`) rather than `tempfile`'s default specifically so the stale sweep is not inert.
 - The route was wired rather than deferred, because deferring would have left `RepositoryReadOnly` reachable only from a test.
-

Verification

`buildlock cargo test -p git-vista-server` → **1064 passed, 0 failed**; `buildlock cargo clippy -p git-vista-server --all-targets -- -D warnings` → clean. 30 of those tests are `preview::suite`.

Mutations were run by hand (patch → run → restore), because `failure-atlas`'s `mutation_check` clones **HEAD** and none of these files are committed yet. Every pair below was executed and the stated verdict observed:

Property	Mutation 1 (removes)	Mutation 2 (weakens)	Result
A2 — nothing written, nothing left	delete the <code>alternates</code> write	<code>mem::forget</code> the <code>TempDir</code>	caught; different assertions — 1 fails at <code>expect_graph (CheckFailed)</code> , 2 passes the object/ref checks and fails only on the surviving <code>gv-preview-j9LoII</code>
A3 — a conflict is an answer	classify exit 1 as Clean	read past <code>-z</code> 's empty record	caught; 1 returns a <code>Graph</code> , 2 reports <code>atab.txt</code> and <code>atab.txt\n</code> beside the real path

Property	Mutation 1 (removes)	Mutation 2 (weakens)	Result
A4 — Unsupported is the default	<code>_ => Some(Merge)</code>	<code>operation_name</code> → constant	caught; 1 draws a graph for a rebase, 2 keeps the arm and fails the literal name
A5 — the merge matches reality	drop the second parent	transpose the parents	caught; 1 fails on row 0's commit, 2 on row 0's parent order
Object format	drop <code>--object-format</code>	hardcode <code>sha256</code>	caught; 1 fails the SHA-256 test, 2 passes it and takes seven SHA-1 tests down instead
The route runs nothing	<code>preview_plan</code> calls <code>plan_and_execute</code>	the needle stops matching a real call	caught; different assertion lines in <code>contract_suite</code>

One mutation survived and the test was fixed. The first version of `parse_merge_tree_conflicts`' suite asserted the stop-at-the-empty-record rule against a fixture whose informational records were `1`, `c.txt`, `Auto-merging`, `Auto-merging c.txt` — **none of which contains a tab** — so replacing `break` with `continue` changed nothing and the test stayed green. The fixture was replaced with git 2.43.0's real output for a file named `has<TAB>atab.txt`, measured on this host, where every

informational record carries a tab. Both mutations then fail, differently. The mechanism was never wrong; the test could not express the failure it claimed to pin.

Not verified, and named rather than claimed:

- `GitTooOld` end to end. The probe is process-cached and this host runs 2.43.0, so only the pure `version_gate` is pinned — with literal versions on both sides of the floor.
- The criss-cross/virtual-base path that motivates `merge_base: None`. No fixture in the catalogue has two merge bases.
- A detached HEAD.
- `POST /api/preview` over real HTTP. The route registration, its authorization classification and its inverse requirement are pinned by source scans; no test drives the endpoint through the router.
- A6, the web canvas. Not in this slice; no frontend file was touched.

Signed: max · 2026-08-30